Software Requirements Specification (SRS) for Intellipath

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to document the functional and non-functional requirements for **Intellipath**, an AI-Powered Personalized Learning Platform. This document serves as the foundation for development, testing, and project management.

1.2 Scope

Intellipath is a cutting-edge educational platform that leverages Artificial Intelligence (AI) to instantly generate personalized curriculum, quizzes, and learning paths for users across any topic.

The system will cover:

- User registration and secure authentication.
- AI-powered instant course and quiz generation using local Large Language Models (LLMs).
- Adaptive learning path management.
- Interactive assessment and knowledge verification.
- Real-time learning analytics and progress tracking.
- A premium, responsive user interface.

1.3 Target Audience

The primary audience for this document includes:

- Development Team (Frontend and Backend Engineers)

- Quality Assurance (QA) Team
- Product Owners/Managers
- Stakeholders

1.4 Technology Stack

Intellipath is a full-stack application built using the following technologies:

Component	Technology	Description
Frontend	Next.js 16 (App Router), TypeScript, Tailwind CSS, Shadcn/UI, Framer Motion	Provides a robust, performant, and premium UI/UX.
Backend	Node.js, Express.js 5, MongoDB (Mongoose)	Provides secure API endpoints and data persistence.
AI Engine	Ollama (Local LLM Integration)	Handles the AI-powered course and quiz generation.
Authentication	JSON Web Tokens (JWT) & Bcrypt	Ensures secure user sessions and password management.

2. Overall Description

2.1 Product Perspective

Intellipath is a standalone, premium learning application designed to operate in a client-server architecture. It interacts with an external, locally hosted AI engine (Ollama) for its core functionality.

2.2 User Classes and Characteristics

User Class	Description	Key Characteristics
Learner	The primary user, seeking personalized education and skill development.	Needs secure login, intuitive navigation, ability to generate courses, take quizzes, and track progress.
Administrator	(Future scope, not detailed here) Manages system settings, user accounts, and	Requires elevated access and administrative tools.

User Class	Description	Key Characteristics
	content if non-AI features are added.	

2.3 Operating Environment

- **Server Environment:** Node.js (v18+), MongoDB, Ollama (running locally with a model pulled, e.g., `llama3`).
- **Client Environment:** Modern web browser (Chrome, Firefox, Edge, Safari) compatible with Next.js 16.
- **Deployment:** Dockerized environment is recommended for consistent setup, although the prerequisites focus on local installation.

3. Functional Requirements

3.1 FR-1: User Authentication and Authorization

Requirement ID	Description
FR-1.1	The system SHALL allow users to register with a unique email and secure password (hashed with Bcrypt).
FR-1.2	The system SHALL allow users to log in using their credentials.
FR-1.3	Successful login SHALL issue a JSON Web Token (JWT) stored as an HTTP-only cookie.
FR-1.4	All protected routes (Client and Server) SHALL be protected by Middleware to verify the JWT.
FR-1.5	The system SHALL provide a secure mechanism for user logout, invalidating the JWT.

3.2 FR-2: AI Course Generation

Requirement ID	Description
FR-2.1	The system SHALL allow the user to input a desired **topic** and a target **skill level** (Beginner, Intermediate, Expert).
FR-2.2	The system SHALL send the topic and skill level to the Ollama (Local LLM) service via the backend.
FR-2.3	The AI service SHALL instantly generate a structured course outline consisting of **Modules** and **Lessons**.
FR-2.4	The generated course SHALL be stored in the MongoDB database, associated with the user.
FR-2.5	The course generation process SHALL include visual feedback (e.g., a loading spinner) while waiting for the Ollama response.

3.3 FR-3: Personalized Learning Paths

Requirement ID	Description
FR-3.1	The system SHALL display the user's generated courses and learning progress on a dedicated dashboard.
FR-3.2	The system SHALL automatically mark a lesson as "Completed" upon successful completion of its associated quiz (see FR-4).
FR-3.3	The curriculum SHALL be adaptive, suggesting the next lesson based on the current progress and the user's initial skill level input.
FR-3.4	Users SHALL be able to manually mark lessons or modules as complete or incomplete.

3.4 FR-4: Interactive Assessments (Quizzes)

Requirement ID	Description
FR-4.1	Upon viewing a lesson, the system SHALL automatically generate an interactive quiz of **3 questions** related to the lesson's content.
FR-4.2	The quiz generation SHALL utilize the Ollama AI engine.
FR-4.3	Quizzes SHALL support multiple-choice questions.
FR-4.4	The system SHALL validate the user's answer and provide immediate feedback (correct/incorrect).
FR-4.5	A user SHALL be prevented from proceeding to the next lesson until the current quiz is successfully completed (e.g., achieving a passing score).

3.5 FR-5: Learning Analytics

Requirement ID	Description
FR-5.1	The system SHALL provide a real-time dashboard displaying key learning analytics.
FR-5.2	The analytics SHALL track and display the total number of lessons completed.
FR-5.3	The analytics SHALL track and display the user's average quiz score across all modules.
FR-5.4	The analytics SHALL track and display the percentage of a course completed.

4. Non-Functional Requirements

4.1 Performance Requirements

- **Responsiveness:** The client application (Frontend) SHALL respond to user interactions (e.g., button clicks, navigation) within 500ms under normal operating conditions.
- **AI Latency:** The AI Course Generation (FR-2) and Quiz Generation (FR-4) should aim to complete the generation process within 10 seconds, provided the Ollama engine and model are correctly configured and running locally.
- **Database:** API query response times for fetching user data and courses SHALL not exceed 300ms.

4.2 Security Requirements

- **Data Protection:** User passwords SHALL be stored as one-way hashes (Bcrypt).
- **Session Management:** Sessions SHALL be managed via JWTs stored in HTTP-only cookies to mitigate Cross-Site Scripting (XSS) attacks.
- **Route Protection:** Sensitive API endpoints and client routes SHALL be secured by authorization middleware to prevent unauthorized access.

4.3 Design and User Experience (UX) Requirements

- **Aesthetics:** The user interface SHALL implement the Premium UI/UX built with Shadcn/UI, Framer Motion, and a customized Material Design 3 aesthetic.
- **Usability:** The platform SHALL be intuitive, providing clear guidance for course generation and progress tracking.
- **Responsiveness:** The frontend SHALL be fully responsive and optimized for desktop, tablet, and mobile viewing.

4.4 Development and Maintainability

- **Code Quality:** The codebase SHALL adhere to TypeScript standards and use clear module separation as outlined in the project structure (Mongoose Schemas, Express Controllers, Next.js components).
- **Installation:** Installation and configuration SHALL be clearly documented, requiring only the prerequisites to be installed. The `Installation` and `Configuration` sections from the project's README are the definitive guide for setup.

5. Prerequisites and Installation Summary

The following table summarizes the required prerequisites for running the Intellipath application.

Component	Prerequisite	Notes
Runtime	Node.js (v18+)	Required for both client and server.
Database	MongoDB	Local instance or Atlas URI (defined in `server/.env`).
AI Engine	Ollama	Must be running locally (e.g., `ollama serve`) with a model (e.g., `llama3`) pulled.

Installation steps:

1. Clone the repository: `git clone https://github.com/viwek-raj/intellipath.git`
2. Install server dependencies: `cd server` then `npm install`
3. Install client dependencies: `cd ../client` then `npm install`
4. Configure the `server/.env` file with `MONGO_URI`, `JWT_SECRET`, and `OLLAMA_HOST`.

To run the project, the following three processes must be started:

1. **Backend:** `cd server` then `npm run dev`
2. **Frontend:** `cd client` then `npm run dev`
3. **AI Engine:** `ollama serve` Calendar event